

7. Test Scenarios and Test Cases

7.1 Overview of Test Scenarios and Test Cases

This section defines the **test scenarios and test cases** designed for validating the ReqRes REST APIs using Postman. A test scenario represents a high-level condition or functionality to be tested, while a test case provides detailed step-by-step validation including inputs, expected results, and actual behavior.

The main objective of this section is to ensure that all API endpoints are tested in both **positive and negative conditions** to verify correctness, reliability, and robustness of the system.

7.2 Login API Test Scenarios

Scenario 1: Verify successful login with valid credentials

Test Case 1.1

- **API:** Login API
- **Method:** POST
- **Endpoint:** `{{base_url}}/login`

Request Body

```
{
  "email": "eve.holt@reqres.in",
  "password": "cityslicka"
}
```

Expected Result

- Status Code: **200 OK**
- Response contains **token**

Actual Validation

- Token should be present in response body
 - Login should be successful
-

Scenario 2: Verify login with invalid credentials

Test Case 1.2

- **API:** Login API
- **Method:** POST

Request Body

```
{  
  "email": "wrong@email.com",  
  "password": "wrongpass"  
}
```

Expected Result

- Status Code: **400 Bad Request**
- Error message displayed

Validation

- System should not allow login
 - Proper error response should be returned
-

7.3 Get Users API Test Scenarios

Scenario 3: Verify retrieval of all users

Test Case 2.1

- **API:** Get Users
- **Method:** GET
- **Endpoint:** `{{base_url}}/users?page=2`

Expected Result

- Status Code: **200 OK**
- List of users returned

Validation

- Response should contain:
 - page number
 - user data array
 - total users
-

Scenario 4: Verify pagination functionality

Test Case 2.2

- Change page number in request

Expected Result

- Different set of users should be returned per page

Validation

- Ensure data changes with page number
-

7.4 Get Single User API Test Scenarios

Scenario 5: Verify valid user retrieval

Test Case 3.1

- **API:** Get Single User
- **Method:** GET
- **Endpoint:** `{{base_url}}/users/2`

Expected Result

- Status Code: **200 OK**
- User details returned

Validation

- Validate user ID, email, first name, last name
-

Scenario 6: Verify invalid user retrieval

Test Case 3.2

- **Endpoint:** `{{base_url}}/users/23`

Expected Result

- Status Code: **404 Not Found**

Validation

- API should return proper error response
-

7.5 Create User API Test Scenarios

Scenario 7: Verify user creation with valid data

Test Case 4.1

- **API:** Create User
- **Method:** POST
- **Endpoint:** `{{base_url}}/users`

Request Body

```
{
  "name": "Tanhaji",
  "job": "QA Engineer"
}
```

Expected Result

- Status Code: **201 Created**
- User ID generated

Validation

- Response should contain:
 - name
 - job
 - id
 - createdAt timestamp
-

7.6 Update User API Test Scenarios

Scenario 8: Verify full update using PUT

Test Case 5.1

- **API:** Update User (PUT)
- **Endpoint:** `{{base_url}}/users/2`

Request Body

```
{
  "name": "Updated Name",
  "job": "Senior QA"
}
```

Expected Result

- Status Code: **200 OK**
 - Data fully updated
-

Scenario 9: Verify partial update using PATCH

Test Case 5.2

- **API:** Update User (PATCH)

Request Body

```
{
  "job": "Lead QA Engineer"
}
```

Expected Result

- Status Code: **200 OK**
 - Only job field updated
-

7.7 Delete User API Test Scenarios

Scenario 10: Verify user deletion

Test Case 6.1

- **API:** Delete User
- **Method:** DELETE
- **Endpoint:** `{{base_url}}/users/2`

Expected Result

- Status Code: **204 No Content**

Validation

- No response body should be returned
 - User should be considered deleted
-

7.8 Negative Test Scenarios Summary

Scenario	API	Expected Result
Invalid login	Login API	400 error
Invalid user ID	Get User	404 error
Wrong data input	Create User	Validation error

7.9 Test Coverage Summary

The following areas are covered in testing:

- Authentication testing (Login API)
 - Positive data validation
 - Negative testing scenarios
 - CRUD operations validation
 - Pagination testing
 - Status code validation
 - Response structure validation
-

7.10 Final Conclusion

The test scenarios and test cases ensure complete validation of ReqRes APIs. Both positive and negative test cases are included to verify system behavior under different conditions.

This structured approach helps ensure:

- API reliability
 - Data accuracy
 - Proper error handling
 - Real-world QA testing coverage
-